Claude Certified Architect — Certificazione Foundations

> **Aggiornamento:** questa guida è aggiornata a **ottobre 2026**. È prevista una revisione a **dicembre 2026**.

Introduzione

La certificazione **Claude Certified Architect — Foundations** attesta che uno specialista è in grado di prendere decisioni ponderate sui compromessi progettuali (trade-off) durante l'implementazione di soluzioni reali basate su Claude. L'esame valuta le conoscenze fondamentali di Claude Code, del Claude Agent SDK, della Claude API e del Model Context Protocol (MCP), le tecnologie principali per la realizzazione di applicazioni di produzione con Claude.

Le domande d'esame si basano su scenari realistici del settore: sviluppo di sistemi agentici per l'assistenza clienti, progettazione di pipeline di ricerca multi-agente, integrazione di Claude Code nei processi CI/CD, creazione di strumenti per aumentare la produttività degli sviluppatori ed estrazione di dati strutturati da documenti non strutturati.

Candidato Ideale

Il candidato ideale è un **solution architect** che progetta e rilascia applicazioni in produzione basate su Claude. È consigliata un'esperienza pratica di almeno 6 mesi nei seguenti ambiti:

- **Claude Agent SDK** — orchestrazione multi-agente, delega a sottoagenti, integrazione di strumenti (tool), hook del ciclo di vita (lifecycle hooks)
- **Claude Code** — CLAUDE.md, server MCP, Agent Skills, modalità di pianificazione (planning mode)
- **Model Context Protocol (MCP)** — strumenti e risorse per l'integrazione con sistemi backend
- **Prompt engineering** — schemi JSON, esempi few-shot, modelli per l'estrazione dei dati
- **Finestre di contesto (context windows)** — gestione di documenti lunghi e passaggio del contesto tra più agenti
- **Pipeline CI/CD** — revisione automatizzata del codice e generazione di test
- **Gestione delle escalation e affidabilità** — gestione degli errori e approcci human-in-the-loop

---

Formato dell'Esame

Parametro	Valore
Tipo di domanda	Scelta multipla (1 risposta corretta su 4)
Punteggio	Scala da 100 a 1000, punteggio minimo per il superamento **720**
Penalità per risposta errata	Nessuna (rispondi a tutte le domande!)
Scenari	4 su 6 possibili (selezionati casualmente)

Contenuto dell'Esame: 5 Domini

Dominio	Peso
1. Architettura e orchestrazione degli agenti	**27%**
2. Progettazione di strumenti e integrazione MCP	**18%**
3. Configurazione e workflow di Claude Code	**20%**
4. Prompt engineering e output strutturato	**20%**
5. Gestione del contesto e affidabilità	**15%**

Tipologia dell’esame

Valutazione dell’esame

L’esame Claude Certified Architect – Foundations è una valutazione basata su criteri: ogni candidato viene confrontato con uno standard di prestazione fisso, non con gli altri candidati. Per superarlo occorre dimostrare le conoscenze e le competenze definite nel blueprint, non ottenere un risultato superiore a una determinata percentuale degli altri partecipanti.

Soglia di superamento. Il punteggio minimo è stato stabilito tramite uno studio formale, nel quale esperti qualificati hanno definito il livello di prestazione atteso da un candidato minimamente qualificato. Il punteggio viene riportato su una scala ponderata da 100 a 1.000; la soglia di superamento è **720**. I modelli di punteggio ponderato consentono di confrontare i risultati di versioni diverse dell’esame, che potrebbero avere livelli di difficoltà leggermente differenti.

Comunicazione dei risultati. Il risultato indica se l’esame è stato superato, insieme a un punteggio ponderato da 100 a 1.000. Il rapporto mostra inoltre la percentuale di domande corrette per ogni dominio. Le percentuali per sezione aiutano a comprendere la propria prestazione, ma non determinano l’esito: il superamento dipende dal punteggio ponderato totale.

Registrazione e prenotazione

La registrazione e la prenotazione avvengono tramite Anthropic Partner Academy e Pearson VUE:

1. Vai alla pagina di certificazione dell’esame su Anthropic Partner Academy e consulta i dettagli.
2. Scarica la guida all’esame e leggi i Termini e condizioni della certificazione e le Regole per gli esami di certificazione prima di registrarti (che trovi nei riferimenti).
3. Registrati all’esame e completa il pagamento. L’importo mostrato al checkout tiene conto degli eventuali sconti previsti dal tuo livello di partner.
4. Segui le istruzioni di conferma per creare un account Pearson VUE, quindi accedi per prenotare la sessione d’esame.
5. Scegli una data disponibile e seleziona la sorveglianza online oppure un centro d’esame Pearson. L’esame è erogato da Pearson VUE.
6. Puoi annullare o riprogrammare l’esame fino a 24 ore prima dell’appuntamento. Le modifiche effettuate nelle 24 ore precedenti comportano la perdita della quota d’esame.

Regole dell’esame

Documento d'identità

Il giorno dell'esame devi presentare un documento d'identità valido, non scaduto, rilasciato da un ente governativo e munito di fotografia. Il nome sul documento deve corrispondere esattamente a quello della registrazione. Per correggere il nome registrato, contatta certifications-support@anthropic.com prima di prenotare l'esame.

Misure di accomodamento

Sono disponibili misure di accomodamento ragionevoli per i candidati con disabilità o esigenze documentate, in conformità alla legge applicabile. Pearson VUE deve approvare le richieste prima della prenotazione. Non prenotare l'appuntamento finché la richiesta non è stata approvata. Le richieste si presentano su pearsonvue.com/us/en/test-takers/accommodations.

Politica sui tentativi successivi

I candidati che non superano l'esame possono ripeterlo dopo un periodo di attesa obbligatorio, che aumenta a ogni tentativo non superato: 14 giorni dopo il primo, 30 giorni dopo il secondo e 90 giorni dopo il terzo. È possibile sostenere lo stesso esame fino a quattro volte nell'arco di dodici mesi. I limiti si applicano a ciascun esame: non superare un esame non impedisce di iscriversi a un esame diverso. La quota è dovuta per ogni tentativo.

Assenza e ritardo

Chi non si presenta all'esame prenotato o arriva oltre il periodo di tolleranza consentito perde la quota d'esame e deve registrarsi nuovamente. Le scadenze per annullare o riprogrammare sono descritte nella sezione Registrazione e prenotazione.

Svolgimento dell'esame e regole di condotta

L'esame si svolge in un ambiente standardizzato, sicuro e sorvegliato. Le regole seguenti valgono sia per gli esami online sia per quelli presso un centro Pearson VUE e tutelano l'integrità della certificazione per tutti i titolari.

Durante l'esame è necessario:

- rimanere sempre nel campo visivo del supervisore e della webcam, se l'esame è online;
- tenere l'area di lavoro libera da appunti, libri, telefoni, monitor secondari e altri materiali;
- non comunicare con altre persone;
- non acquisire, copiare, fotografare o riprodurre in alcun modo i contenuti dell'esame.

Tra gli oggetti vietati rientrano telefoni cellulari, smartwatch, cuffie, materiali di studio e dispositivi di registrazione. Gli eventuali oggetti consentiti, come la carta per appunti fornita dal supervisore, sono specificati da Pearson VUE.

Conseguenze della condotta scorretta. Barare, tentare di accedere a risorse vietate o divulgare contenuti dell'esame può comportare l'annullamento del risultato, la revoca della credenziale e l'esclusione da esami futuri.

Riservatezza e accordo di non divulgazione

Prima dell'inizio dell'esame devi accettare un accordo di riservatezza e non divulgazione. Accettandolo, riconosci che tutti i contenuti dell'esame, comprese domande, opzioni di risposta e scenari, sono informazioni riservate e proprietà esclusiva di Anthropic e accetti di non divulgarli, riprodurli o distribuirli. Se non accetti l'accordo, la sessione termina e non viene emesso alcun rimborso.

Mantenimento della credenziale e ricertificazione

La credenziale Claude Certified Architect – Foundations è valida per 12 mesi dalla data di assegnazione. Poiché la tecnologia sottostante evolve rapidamente, la credenziale ha una durata limitata affinché i titolari mantengano aggiornate le proprie conoscenze.

Per rinnovarla entro i termini, verifica le modifiche intervenute dalla certificazione e completa una valutazione gratuita, non sorvegliata, su Anthropic Partner Academy. Il rinnovo puntuale non prevede costi. Se la credenziale scade, per riottenere lo stato di certificato devi ripetere l'esame completo pagando la quota intera.

Se i contenuti dell'esame cambiano in modo significativo, Anthropic può richiedere ai titolari di ripetere l'esame completo per ricertificarsi, anziché completare la valutazione di rinnovo. I titolari restano soggetti alle regole di condotta descritte nella sezione Svolgimento dell'esame e regole di condotta.

Scenari d'Esame

Ogni domanda d'esame si apre con un **testo introduttivo fisso** che descrive lo scenario. È sempre lo stesso (con piccole variazioni di parole) per tutte le domande di quello scenario, ed è in inglese. Riconoscerlo al volo fa risparmiare tempo: lo leggi velocemente, capisci subito di quale contesto si parla e ti concentri sulla parte nuova della domanda, che viene dopo.

Per ogni scenario trovi il testo così come compare nelle domande, la traduzione e una breve spiegazione di cosa ti sta dicendo.

Scenario 1: Agente per l'assistenza al cliente

Viene costruito un agente per gestire resi, contestazioni di fatturazione e problemi relativi agli account utilizzando il Claude Agent SDK. L'agente utilizza strumenti MCP (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). L'obiettivo è raggiungere un tasso di risoluzione al primo contatto superiore all'80%, con escalation appropriata quando necessario.

> **Customer Support Resolution Agent**
>
> You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

Traduzione: stai costruendo un agente per l'assistenza clienti con il Claude Agent SDK. L'agente gestisce richieste molto ambigue come resi, contestazioni di fatturazione e problemi di account. Accede ai sistemi backend tramite strumenti MCP personalizzati. L'obiettivo è risolvere oltre l'80% delle richieste al primo contatto, sapendo quando passare il caso a una persona.

Cosa ti sta dicendo: è lo scenario dell'**affidabilità in produzione**. Gli strumenti hanno un ordine logico (prima identifichi il cliente, poi l'ordine, poi il rimborso) e le regole che "devono" valere (verifica prima del rimborso, limiti di importo) vanno garantite con codice e hook, non con il prompt. L'80% al primo contatto indica che l'escalation va fatta **quando serve davvero**: né troppo spesso né troppo poco, e con un passaggio di consegne completo. Domini più coinvolti: D1, D2, D5.

Scenario 2: Generazione di codice con Claude Code

Claude Code viene utilizzato per accelerare lo sviluppo: generazione di codice, refactoring, debug e documentazione. È necessario integrarlo con comandi personalizzati e configurazione tramite CLAUDE.md, oltre a comprendere quando utilizzare la modalità di planning.

> **Code Generation with Claude Code**
>
> You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, `CLAUDE.md` configurations, and understand when to use plan mode vs direct execution.

Traduzione: usi Claude Code per accelerare lo sviluppo software. Il team lo usa per generare codice, fare refactoring, debug e documentazione. Devi integrarlo nel flusso di lavoro con comandi slash personalizzati e configurazioni `CLAUDE.md`, e capire quando usare la modalità plan e quando l'esecuzione diretta.

Cosa ti sta dicendo: è lo scenario della **configurazione di Claude Code per un team**. Le domande riguardano dove mettere le istruzioni (`CLAUDE.md` di progetto o utente, regole per percorso), cosa è condiviso nel repository e cosa resta personale, comandi e skill. La scelta tra plan ed esecuzione diretta è un classico: plan per modifiche ampie, ambigue o con più approcci possibili; esecuzione diretta per modifiche piccole e chiare. Dominio più coinvolto: D3.

Scenario 3: Sistema di ricerca multi-agente

Un agente coordinatore delega attività a subagenti specializzati: ricerca web, analisi documentale, sintesi e generazione di report. Il sistema deve produrre report completi con citazioni.

> **Multi-Agent Research System**
>
> You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Traduzione: stai costruendo un sistema di ricerca multi-agente con il Claude Agent SDK. Un agente coordinatore delega a subagenti specializzati: uno cerca sul web, uno analizza documenti, uno sintetizza i risultati e uno genera i report. Il sistema fa ricerche su un argomento e produce report completi e con citazioni.

Cosa ti sta dicendo: è lo scenario dell'**orchestrazione**. I subagenti non vedono il contesto del coordinatore: tutto ciò che serve va passato esplicitamente. "Completi" richiama la scomposizione del lavoro (argomenti non coperti, compiti delegati in modo troppo stretto); "con citazioni" richiama la **provenienza** delle informazioni,

che non deve perdersi tra un agente e l'altro. Tipici anche gli errori parziali (un subagente fallisce) e le fonti in conflitto. Domini più coinvolti: D1, D5.

Scenario 4: Strumenti di produttività per sviluppatori

L'agente supporta gli sviluppatori nell'esplorazione di codebase sconosciute, nella generazione di codice boilerplate e nell'automazione di attività ripetitive. Vengono utilizzati strumenti integrati (Read, Write, Bash, Grep, Glob) e server MCP.

> **Developer Productivity with Claude**
>
> You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Traduzione: stai costruendo strumenti di produttività per sviluppatori con il Claude Agent SDK. L'agente aiuta gli ingegneri a esplorare codebase sconosciute, capire sistemi legacy, generare codice ripetitivo e automatizzare attività ricorrenti. Usa gli strumenti integrati (Read, Write, Bash, Grep, Glob) e si integra con server MCP.

Cosa ti sta dicendo: è lo scenario della **scelta degli strumenti giusti**. Devi sapere quale strumento integrato usare per ogni compito (Grep per cercare nel contenuto dei file, Glob per trovare file per nome, Read prima di modificare, Edit invece di riscrivere tutto il file) e quando serve un server MCP. "Codebase sconosciute" e "sistemi legacy" richiamano la gestione del contesto: esplorare in modo mirato invece di leggere tutto. Domini più coinvolti: D2, D3, D5.

Scenario 5: Claude Code per la Continuous Integration

Integrazione di Claude Code in una pipeline CI/CD per revisione automatica del codice, generazione di test e feedback sulle pull request. I prompt devono essere progettati per ridurre al minimo i falsi positivi.

> **Claude Code for Continuous Integration**
>
> You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Traduzione: stai integrando Claude Code nella pipeline CI/CD. Il sistema esegue revisioni automatiche del codice, genera casi di test e fornisce feedback sulle pull request. Devi progettare prompt che diano indicazioni concrete e riducano al minimo i falsi positivi.

Cosa ti sta dicendo: è lo scenario dell'**automazione senza persone davanti allo schermo**. Claude Code gira in modalità non interattiva (`-p`), con output strutturato leggibile da un programma, permessi minimi e segreti presi dalla piattaforma. "Falsi positivi" è la parola chiave: servono criteri di revisione espliciti e specifici, esempi e categorie di problemi precise, non un generico "sii più accurato". Compare spesso anche la scelta tra chiamata sincrona e Batch API. Domini più coinvolti: D3, D4.

Scenario 6: Estrazione di Dati Strutturati

Il sistema estrae informazioni da documenti non strutturati, valida l'output tramite JSON schema e mantiene un'elevata accuratezza. Deve gestire correttamente anche i casi limite.

> **Structured Data Extraction**
>
> You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JSON schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Traduzione: stai costruendo un sistema di estrazione di dati strutturati con Claude. Il sistema estrae informazioni da documenti non strutturati, valida l'output con schemi JSON e mantiene un'alta accuratezza. Deve gestire bene i casi limite e integrarsi con i sistemi a valle.

Cosa ti sta dicendo: è lo scenario dell'**output strutturato e della validazione**. Lo schema garantisce la forma, non la correttezza: servono anche controlli semantici (totali che tornano, date plausibili). "Casi limite" significa campi mancanti (si usa `null`, non si inventa), documenti in formati diversi, valori ambigui. "Sistemi a valle" significa che l'output deve essere sempre leggibile da un programma. Tipici anche few-shot, nuovo tentativo con il messaggio d'errore e Batch API per grandi volumi. Domini più coinvolti: D4, D5.

Scenario 7: Pattern di architettura per AI conversazionale (EXTRA)

Si progettano sistemi conversazionali multi-turno che includono gestione della finestra di contesto, persistenza delle istruzioni tra i turni, strategie di memoria, progettazione di strumenti per l'esecuzione sicura e gestione di input utente ambigui o contraddittori.

Testo nelle domande: a differenza degli altri, questo scenario **non ha un'introduzione fissa**. Le domande vanno subito al caso concreto, per esempio:

> *A conversational application uses server-side compaction, but on every turn it adds only `response.content[0].text` to the history. What is wrong?*

Cosa ti sta dicendo: è lo scenario delle **conversazioni lunghe**. La Claude API non ricorda nulla tra una chiamata e l'altra: la storia va reinviata completa e corretta, inclusi i blocchi dei tool e della compattazione (nell'esempio, salvare solo il testo fa perdere proprio quei blocchi). Le domande riguardano cosa fare quando il contesto cresce (compattazione, riepiloghi, fatti importanti conservati a parte), come far valere le istruzioni per tutta la conversazione e come gestire richieste ambigue o in contraddizione con quanto detto prima. Domini più coinvolti: D1, D5.

Documentazione Ufficiale

Link verificati a ottobre 2026. Dove disponibile si usa la versione italiana della documentazione. Accanto a ogni risorsa trovi i domini d'esame a cui si riferisce.

Claude API

Risorsa	Domini	URL
Messages API (riferimento)	D1, D5	platform.claude.com/docs/it/api/messages

Risorsa	Domini	URL
Uso degli strumenti (tool use)	D1, D2	platform.claude.com/docs/it/agents-and-tools/tool-use/overview
Motivi di arresto (`stop_reason`)	D1, D5	platform.claude.com/docs/it/build-with-claude/handling-stop-reasons
Output strutturati	D4	platform.claude.com/docs/it/build-with-claude/structured-outputs
Elaborazione batch (Message Batches)	D3, D4	platform.claude.com/docs/it/build-with-claude/batch-processing
Cache dei prompt	D4	platform.claude.com/docs/it/build-with-claude/prompt-caching
Prompt engineering	D4	platform.claude.com/docs/it/build-with-claude/prompt-engineering/overview
Ragionamento esteso (extended thinking)	D4	platform.claude.com/docs/it/build-with-claude/extended-thinking
Orientare il ragionamento (thinking adattivo)	D4	platform.claude.com/docs/it/build-with-claude/thinking-steering-and-cost
Effort	D4	platform.claude.com/docs/it/build-with-claude/effort
Finestre di contesto	D5	platform.claude.com/docs/it/build-with-claude/context-windows
Conteggio dei token	D5	platform.claude.com/docs/it/build-with-claude/token-counting
Modifica del contesto (context editing)	D5	platform.claude.com/docs/it/build-with-claude/context-editing
Compattazione	D5	platform.claude.com/docs/it/build-with-claude/compaction
Strumento memoria	D5	platform.claude.com/docs/it/agents-and-tools/tool-use/memory-tool

Claude Agent SDK

Risorsa	Domini	URL
Panoramica	D1	code.claude.com/docs/it/agent-sdk/overview
Hooks	D1	code.claude.com/docs/it/agent-sdk/hooks
Subagents	D1	code.claude.com/docs/it/agent-sdk/subagents
Sessioni	D1	code.claude.com/docs/it/agent-sdk/sessions

Claude Code

Risorsa	Domini	URL
Panoramica	D3	code.claude.com/docs/it/overview
CLAUDE.md, regole (`.claude/rules/`) e memoria	D3	code.claude.com/docs/it/memory
File di impostazioni e precedenza	D1, D3	code.claude.com/docs/it/settings
Permessi e modalità di permesso	D3	code.claude.com/docs/it/permissions
Skills (inclusi i comandi slash)	D3	code.claude.com/docs/it/skills
Hooks (riferimento)	D1, D3	code.claude.com/docs/it/hooks
Subagent personalizzati	D1, D3	code.claude.com/docs/it/sub-agents
MCP in Claude Code	D2	code.claude.com/docs/it/mcp
Flussi di lavoro comuni (plan mode, sessioni, worktree)	D3	code.claude.com/docs/it/common-workflows
Riferimento CLI (flag come `-p`, `--output-format`)	D3	code.claude.com/docs/it/cli-reference
Esecuzione programmatica (headless, `-p`)	D3	code.claude.com/docs/it/headless
GitHub Actions	D3	code.claude.com/docs/it/github-actions
GitLab CI/CD	D3	code.claude.com/docs/it/gitlab-ci-cd
Routines (attività pianificate nel cloud)	D3	code.claude.com/docs/it/routines
Attività pianificate e `/loop`	D3	code.claude.com/docs/it/scheduled-tasks

Model Context Protocol (MCP)

Il sito MCP è in inglese. I link "latest" puntano sempre all'ultima versione della specifica.

Risorsa	Domini	URL
Introduzione	D2	modelcontextprotocol.io
Architettura (host, client, server)	D2	modelcontextprotocol.io/docs/learn/architecture
Concetti dei server (tools, resources, prompts)	D2	modelcontextprotocol.io/docs/learn/server-concepts
Specifica — Tools	D2	modelcontextprotocol.io/specification/latest/server/tools
Specifica — Resources	D2	modelcontextprotocol.io/specification/latest/server/resources
Specifica — Trasporti (stdio, streamable HTTP)	D2	modelcontextprotocol.io/specification/latest/basic/transports

Corsi ed esempi ufficiali

Risorsa	URL
Anthropic Academy (corsi gratuiti)	anthropic.skilljar.com
Claude Cookbooks (esempi di codice)	github.com/anthropics/claude-cookbooks

Risorse di preparazione non ufficiali

Siti di terze parti, non affiliati ad Anthropic: utili per esercitarsi, ma in caso di dubbio fa fede la documentazione ufficiale.

Risorsa	URL
Claude Certification Guide	claudecertificationguide.com
Termidy	claude.termidy.com

> La teoria non è in questa guida: si trova nei documenti dei singoli domini (appunti spiegati). Qui sotto c'è solo la lista degli argomenti per dominio.

Dominio 1: Architettura degli agenti e orchestrazione (27%)

1.1 Progettazione di agentic loop per l'esecuzione autonoma dei task

Conoscenze chiave

- Ciclo di vita dell'agent loop: inviare una richiesta a Claude, verificare `stop_reason` (`"tool_use"` vs `"end_turn"`), eseguire i tool, reinserire i risultati per l'iterazione successiva
- I risultati dei tool vengono aggiunti alla history della conversazione, consentendo al modello di decidere l'azione successiva
- Decisione guidata dal modello (Claude seleziona il prossimo tool) vs logiche deterministiche hard-coded

Competenze chiave

- Controllo del flusso: continuare il loop quando `stop_reason = "tool_use"` e terminare su `"end_turn"`
- Aggiunta dei risultati dei tool al contesto tra un'iterazione e la successiva
- Anti-pattern da evitare: parsing del testo dell'assistant per determinare la fine, uso di limiti arbitrari di iterazioni come meccanismo primario di stop

1.2 Orchestrazione di sistemi multi-agent (Coordinator–Subagent)

Conoscenze chiave

- Architettura hub-and-spoke: il coordinator gestisce tutta la comunicazione tra agenti, l'error handling e il routing
- I subagent operano con contesto isolato: non ereditano automaticamente la history del coordinator

- Responsabilità del coordinator: decomposizione del task, delega, aggregazione dei risultati, selezione dinamica dei subagent
- Rischio di decomposizione eccessivamente frammentata da parte del coordinator

Competenze chiave

- Suddividere la copertura di ricerca tra subagent per minimizzare duplicazioni
- Implementare loop di raffinamento iterativo (il coordinator valuta la sintesi e rialloca i task)
- Instradare tutta la comunicazione attraverso il coordinator per garantire osservabilità

1.3 Configurazione delle chiamate ai subagent, context passing e spawning

Conoscenze chiave

- Il tool `Task` permette di spawnare subagent; `allowedTools` del coordinator deve includere `"Task"`
- Il contesto del subagent deve essere passato esplicitamente nel prompt; non esiste inheritance automatica
- Configurazione `AgentDefinition`: descrizioni, system prompt, vincoli sui tool
- Gestione sessioni tramite `fork_session` per esplorare alternative

Competenze chiave

- Includere output completi di agent precedenti nel prompt del subagent
- Usare formati strutturati per separare dati e metadata nel context passing
- Avviare subagent in parallelo tramite più chiamate `Task` in una singola iterazione del coordinator
- Scrivere prompt del coordinator orientati a obiettivi e criteri di qualità, non a istruzioni passo-passo

1.4 Implementazione di workflow multi-step con enforcement e pattern di handoff

Conoscenze chiave

- Differenza tra **enforcement programmatico** (hooks, precondizioni) e guida tramite prompt per l'ordinamento di un workflow
- Quando sono necessarie garanzie deterministiche (es. verifica identità prima di operazioni finanziarie), i prompt non sono sufficienti
- Protocolli strutturati di handoff durante l'escalation (customer ID, motivo, azione raccomandata)

Competenze chiave

- Precondizioni programmatiche che bloccano le chiamate downstream finché gli step precedenti non sono completati (es. bloccare `process_refund` finché `get_customer` non restituisce un ID verificato)
- Scomposizione di richieste multi-aspetto del cliente in elementi separati
- Produzione di summary strutturati durante l'escalation verso un operatore umano

1.5 Hook dell'Agent SDK per intercettazione chiamate tool e normalizzazione dati

Conoscenze chiave

- Pattern di hook (es. `PostToolUse`) per intercettare i risultati dei tool prima che vengano consumati dal modello
- Hook che intercettano chiamate in uscita per imporre regole di compliance (es. blocco rimborsi sopra una soglia)
- Gli hook forniscono **garanzie deterministiche**, mentre le istruzioni nei prompt forniscono **compliance probabilistica**

Competenze chiave

- `PostToolUse` hook per normalizzare formati dati (timestamp Unix, ISO 8601, codici numerici di stato)
- Hook di intercettazione per bloccare azioni non conformi alle policy e reindirizzare verso escalation
- Uso degli hook al posto dei prompt quando le regole di business richiedono enforcement garantito

1.6 Strategie di decomposizione dei task per workflow complessi

Conoscenze chiave

- **Pipeline fisse** (prompt chaining) vs **decomposizione dinamica adattiva** basata su risultati intermedi
- Prompt chaining: sequenze deterministiche (analisi file per file, poi fase di integrazione)
- Piani di investigazione adattivi che generano sotto-task in base ai risultati emersi

Competenze chiave

- Usare prompt chaining per review multi-aspetto prevedibili; usare decomposizione dinamica per investigazioni aperte
- Suddividere code review ampie in analisi per file + passaggio finale di integrazione cross-file
- Decomporre task esplorativi: prima mappare la struttura, poi costruire un piano prioritizzato

1.7 Stato di sessione, resume e fork

Conoscenze chiave

- `--resume <session-name>` per continuare sessioni nominate
- `fork_session` per creare branch indipendenti di investigazione a partire da un contesto condiviso
- Importanza di informare l'agente quando i file sono cambiati durante il resume di una sessione
- Una nuova sessione con un riepilogo strutturato può essere più affidabile rispetto al ripristino di una sessione con risultati obsoleti

Competenze chiave

- Usare `--resume` per continuare sessioni di investigazione nominate
- Usare `fork_session` per confrontare approcci in parallelo
- Scegliere tra resume (contesto ancora valido) e nuova sessione (risultati obsoleti o degradati)

Dominio 2: Progettazione degli Strumenti e Integrazione MCP (18%)

2.1 Progettare Interfacce di Strumenti con Descrizioni Chiare

Conoscenze chiave

- Le descrizioni degli strumenti sono il **meccanismo principale** che un LLM utilizza per selezionare gli strumenti; descrizioni minimali portano a una selezione poco affidabile.
- L'importanza di includere formati di input, esempi di query, casi limite ed eventuali limiti di applicabilità.
- Descrizioni ambigue o sovrapposte causano instradamenti errati.
- La formulazione del prompt di sistema può creare associazioni indesiderate con determinati strumenti.

Competenze chiave

- Scrivere descrizioni che distinguano chiaramente ogni strumento da alternative simili.
- Rinominare gli strumenti per eliminare sovrapposizioni funzionali (ad esempio, `analyze_content` → `extract_web_results`).
- Suddividere strumenti generici in strumenti specializzati con contratti di input/output ben definiti.

2.2 Implementare Risposte di Errore Strutturate per gli Strumenti MCP

Conoscenze chiave

- Il flag `isError` nelle risposte degli strumenti MCP.
- La differenza tra **errori transitori** (timeout), **errori di validazione** (input non valido), **errori di business** (violazioni delle policy) ed **errori di accesso/autorizzazione**.
- Errori generici ("Operazione non riuscita") impediscono di prendere decisioni corrette sul recupero dell'errore.
- La differenza tra errori recuperabili tramite nuovo tentativo (retryable) e non recuperabili.

Competenze chiave

- Restituire metadati strutturati come `errorCategory` (transient/validation/permission), `isRetryable` e un messaggio leggibile dall'utente.
- Utilizzare `retryable: false` per violazioni delle regole di business, accompagnandolo con spiegazioni chiare rivolte all'utente.
- Gestire il recupero locale all'interno dei sottoagenti in caso di errori transitori; propagare solo gli errori che non possono essere risolti autonomamente.
- Distinguere i problemi di accesso (che influenzano la decisione di effettuare un nuovo tentativo) dai risultati validamente vuoti (nessuna corrispondenza trovata).

2.3 Assegnare gli Strumenti agli Agenti e Configurare `tool_choice`

Conoscenze chiave

- Un numero eccessivo di strumenti per agente (ad esempio 18 invece di 4–5) **riduce** l'affidabilità della selezione degli strumenti.
- Gli agenti che dispongono di strumenti al di fuori della propria specializzazione tendono a utilizzarli in modo improprio.
- Accesso limitato agli strumenti (scoped tool access): solo strumenti pertinenti al ruolo, più un insieme ristretto di utility condivise tra ruoli.

- `tool_choice`: `"auto"`, `"any"` e selezione forzata di uno strumento (`{"type": "tool", "name": "..."}`).

Competenze chiave

- Limitare il set di strumenti di ogni sottoagente a quelli rilevanti per il suo ruolo.
- Sostituire strumenti generici con alternative più vincolate (ad esempio, `fetch_url` → `load_document`).
- Utilizzare `tool_choice: "any"` per garantire una chiamata a uno strumento invece di ottenere una risposta testuale.
- Forzare l'uso di uno strumento specifico per garantire un determinato ordine di esecuzione.

2.4 Integrare Server MCP in Claude Code e nei Flussi di Lavoro degli Agenti

Conoscenze chiave

- Ambito dei server MCP: progetto (`.mcp.json`) per i team, utente (`~/.claude.json`) per esperimenti personali.
- Sostituzione delle variabili d'ambiente in `.mcp.json` (ad esempio `${GITHUB_TOKEN}`) per la gestione dei segreti e delle credenziali.
- Gli strumenti provenienti da tutti i server MCP connessi vengono rilevati al momento della connessione e sono disponibili simultaneamente.
- Le risorse MCP come "cataloghi di contenuti" (riepiloghi di attività, schemi di database) riducono la necessità di chiamate esplorative agli strumenti.

Competenze chiave

- Configurare server MCP condivisi nel file `.mcp.json` del progetto utilizzando token basati su variabili d'ambiente.
- Mantenere i server personali o sperimentali nel file `~/.claude.json`.
- Preferire server MCP della community rispetto a server personalizzati per le integrazioni standard.

2.5 Selezionare e Utilizzare gli Strumenti Integrati (Read, Write, Edit, Bash, Grep, Glob)

Conoscenze chiave

- **Grep**: ricerca all'interno del contenuto dei file (nomi di funzioni, messaggi di errore, importazioni).
- **Glob**: ricerca di file tramite pattern di nomi o estensioni.
- **Read/Write**: operazioni sull'intero file; **Edit**: modifiche mirate tramite corrispondenze di testo univoche.
- Se Edit fallisce a causa di corrispondenze non univoche, utilizzare Read + Write come alternativa.

Competenze chiave

- Utilizzare Grep per la ricerca nei contenuti e Glob per individuare file tramite pattern.
- Costruire la comprensione del codice in modo incrementale: usare prima Grep per individuare i punti di ingresso, poi Read per seguire il flusso di esecuzione.
- Tracciare l'utilizzo delle funzioni attraverso moduli wrapper.

Dominio 3: Configurazione di Claude Code e Flussi di Lavoro (20%)

3.1 Configurare CLAUDE.md con Gerarchia, Ambito e Organizzazione Modulare

Conoscenze chiave

- Gerarchia di CLAUDE.md: livello utente (`~/.claude/CLAUDE.md`), livello progetto (`.claude/CLAUDE.md` oppure `CLAUDE.md` nella root del progetto) e livello directory (file `CLAUDE.md` nelle sottodirectory).
- Le impostazioni a livello utente si applicano soltanto a un singolo utente e non vengono condivise tramite il sistema di controllo versione (VCS).
- Sintassi `@path` per fare riferimento a file esterni (ad esempio `@./standards/coding-style.md`) e modularizzare il contenuto di CLAUDE.md.
- La directory `.claude/rules/` per organizzare regole specifiche per argomento invece di utilizzare un unico CLAUDE.md monolitico.

Competenze chiave

- Diagnosticare problemi di gerarchia (ad esempio, un nuovo membro del team non riceve determinate istruzioni perché sono definite a livello utente anziché a livello progetto).
- Utilizzare `@path` (ad esempio `@./standards/testing.md`) per includere selettivamente gli standard appropriati nel file CLAUDE.md di ciascun pacchetto.
- Suddividere un CLAUDE.md di grandi dimensioni in più file all'interno di `.claude/rules/` (ad esempio `testing.md`, `api-conventions.md`, `deployment.md`).

3.2 Creare e Configurare Comandi Slash Personalizzati e Skill

Conoscenze chiave

- **Comandi di progetto** nella directory `.claude/commands/` (condivisi tramite VCS) rispetto ai **comandi utente** in `~/.claude/commands/`.
- Skill archiviate in `.claude/skills/` con frontmatter in `SKILL.md`: `context: fork`, `allowed-tools`, `argument-hint`.
- `context: fork` esegue la skill in un contesto isolato di sottoagente, evitando che influenzi o "inquini" la sessione principale.
- Varianti personali delle skill possono essere conservate in `~/.claude/skills/` con nomi differenti.

Competenze chiave

- Salvare i comandi slash di progetto nella directory `.claude/commands/` affinché siano disponibili a tutto il team.
- Utilizzare `context: fork` per isolare skill che producono output molto verboso.
- Utilizzare `allowed-tools` per limitare gli strumenti che una skill può utilizzare.
- Utilizzare `argument-hint` per richiedere agli sviluppatori i parametri necessari.

3.3 Utilizzare Regole Specifiche per Percorso per il Caricamento Condizionale delle Convenzioni

Conoscenze chiave

- I file presenti in `.claude/rules/` possono includere un frontmatter YAML con il campo `paths` per attivare regole in base a pattern glob.
- Le regole limitate a specifici percorsi vengono caricate **solo** quando si modificano file corrispondenti, riducendo il consumo di contesto e token.
- Le regole basate su pattern glob possono essere preferibili rispetto a CLAUDE.md a livello di directory quando le convenzioni si applicano a molte directory diverse (ad esempio ai test).

Competenze chiave

- Creare file in `.claude/rules/` con `paths: ["terraform/**/*"]` per caricare le regole solo quando si lavora su file corrispondenti.
- Utilizzare pattern glob (ad esempio `**/*.test.tsx`) per applicare convenzioni in base al tipo di file indipendentemente dalla loro posizione.
- Preferire regole specifiche per percorso rispetto a CLAUDE.md a livello di directory quando le convenzioni si estendono a tutto il codebase.

3.4 Decidere Quando Utilizzare la Modalità di Pianificazione rispetto all'Esecuzione Diretta

Conoscenze chiave

- **Modalità di pianificazione (Planning mode)**: adatta a compiti complessi che comportano modifiche estese, più approcci possibili e decisioni architetturali.
- **Esecuzione diretta (Direct execution)**: adatta a modifiche semplici e ben comprese (ad esempio l'aggiunta di una singola validazione).
- La modalità di pianificazione consente di esplorare in sicurezza il codebase prima di apportare modifiche.
- Il sottoagente Explore isola l'output dettagliato dell'attività di esplorazione.

Competenze chiave

- Utilizzare la modalità di pianificazione per attività con implicazioni architetturali (microservizi, migrazioni che coinvolgono oltre 45 file).
- Utilizzare l'esecuzione diretta per correzioni supportate da uno stack trace chiaro e che coinvolgono un solo file.
- Utilizzare il sottoagente Explore per evitare l'esaurimento della finestra di contesto nelle attività suddivise in più fasi.
- Combinare i due approcci: pianificare la fase di analisi ed esplorazione, quindi eseguire l'implementazione.

3.5 Perfezionamento Iterativo per il Miglioramento Progressivo

Conoscenze chiave

- Esempi concreti di input e output rappresentano il modo più efficace per comunicare le aspettative.
- **Iterazione guidata dai test (Test-driven iteration)**: scrivere prima i test e poi iterare in base agli errori e ai fallimenti riscontrati.
- Il modello dell'"intervista" (interview pattern): Claude pone domande per far emergere considerazioni progettuali non immediatamente evidenti.

- Quando presentare tutti i problemi in un unico messaggio (se interdipendenti) rispetto a presentarli in modo sequenziale (se indipendenti).

Competenze chiave

- Fornire 2–3 esempi concreti di input/output per chiarire i requisiti di una trasformazione.
- Costruire set di test che includano comportamento atteso, casi limite e requisiti prestazionali prima dell'implementazione.
- Utilizzare il modello dell'intervista per far emergere aspetti progettuali rilevanti (invalidazione della cache, modalità di errore, ecc.).
- Fornire casi di test concreti con input di esempio e output attesi per i casi limite.

3.6 Integrare Claude Code nelle Pipeline CI/CD

Conoscenze chiave

- Il flag `-p` (oppure `--print`) per l'esecuzione in modalità non interattiva all'interno di pipeline automatizzate.
- `--output-format json` e `--json-schema` per ottenere output strutturati nei processi CI.
- CLAUDE.md fornisce il contesto del progetto (standard di testing, criteri di revisione) alle esecuzioni di Claude Code avviate dalla CI.
- **Isolamento del contesto di sessione**: la stessa sessione che ha generato il codice è generalmente meno efficace nel revisionarlo rispetto a un'istanza indipendente.

Competenze chiave

- Eseguire Claude Code in CI utilizzando `-p` per evitare blocchi dovuti alla richiesta di input interattivi.
- Utilizzare `--output-format json` insieme a `--json-schema` per produrre risultati strutturati (ad esempio commenti inline nelle pull request).
- Includere i risultati delle revisioni precedenti quando si riesegue l'analisi dopo nuovi commit (segnalando solo i problemi nuovi o non ancora risolti).
- Documentare in CLAUDE.md gli standard di testing e i fixture disponibili per migliorare la qualità della generazione dei test.
- Includere nel contesto i file di test esistenti quando si generano nuovi test, per evitare duplicazioni e mantenere uno stile coerente.

Dominio 4: Prompt Engineering e Output Strutturato (20%)

4.1 Progettare Prompt con Criteri Espliciti per Migliorare l'Accuratezza

Conoscenze chiave

- I criteri espliciti sono più efficaci delle istruzioni vaghe (ad esempio, “segnala i commenti solo quando contraddicono il codice” invece di “verifica l'accuratezza dei commenti”).
- Indicazioni generiche come “sii più conservativo” funzionano peggio rispetto a criteri concreti e categorizzati.

- L'effetto dei falsi positivi sulla fiducia degli sviluppatori: tassi elevati di falsi positivi in alcune categorie compromettono la fiducia anche nelle categorie in cui le segnalazioni sono accurate.

Competenze chiave

- Definire criteri di revisione chiari: cosa segnalare (bug, vulnerabilità di sicurezza) e cosa ignorare (piccoli problemi di stile).
- Disattivare temporaneamente le categorie che presentano un alto tasso di falsi positivi.
- Definire criteri di gravità espliciti corredati da esempi di codice per ciascun livello.

4.2 Utilizzare il Few-shot Prompting per Migliorare la Coerenza dell'Output

Conoscenze chiave

- Gli esempi few-shot rappresentano il metodo più efficace per ottenere output coerenti, formattati correttamente e immediatamente utilizzabili.
- Il few-shot può mostrare come gestire casi ambigui (selezione degli strumenti, lacune nella copertura dei test).
- Il few-shot aiuta il modello a generalizzare nuovi schemi invece di limitarsi a ripetere comportamenti predefiniti.
- Il few-shot può ridurre le allucinazioni nei compiti di estrazione delle informazioni.

Competenze chiave

- Fornire da 2 a 4 esempi mirati per scenari ambigui, includendo anche la motivazione delle scelte effettuate.
- Includere esempi few-shot che mostrino chiaramente il formato dell'output (posizione, problema, gravità, correzione suggerita).
- Fornire esempi che distinguano pattern di codice accettabili da problemi reali.
- Fornire esempi di estrazione corretta da documenti con strutture differenti.

4.3 Applicare Output Strutturati con `tool_use` e Schemi JSON

Conoscenze chiave

- L'utilizzo di `tool_use` con schemi JSON è il metodo più affidabile per garantire output conformi allo schema ed eliminare errori di sintassi JSON.
- Con `tool_choice: "auto"` il modello può restituire testo libero; con `"any"` deve necessariamente chiamare uno strumento; la selezione forzata impone l'uso di uno strumento specifico.
- Schemi JSON rigorosi eliminano gli errori sintattici, ma non quelli semantici (totali incoerenti, valori inseriti nel campo sbagliato).
- Progettazione degli schemi: campi obbligatori e facoltativi; enum con valori come `"other"` accompagnati da un campo descrittivo per garantire l'estensibilità.

Competenze chiave

- Definire strumenti di estrazione basati su schemi JSON e analizzare i dati restituiti dai risultati di `tool_use`.

- Utilizzare `tool_choice: "any"` per garantire un output strutturato quando sono disponibili più schemi.
- Forzare la chiamata di uno strumento specifico: `tool_choice: {"type": "tool", "name": "extract_metadata"}`.
- Rendere i campi facoltativi o nullable quando la fonte potrebbe non contenere l'informazione, evitando di inventare dati mancanti.
- Utilizzare valori enum come `"unclear"` e `"other"` accompagnati da campi descrittivi per una categorizzazione estendibile.

4.4 Implementare Validazione, Retry e Cicli di Feedback per Migliorare la Qualità dell'Estrazione

Conoscenze chiave

- Retry con feedback sugli errori (retry-with-error-feedback): includere nel prompt di retry errori di validazione concreti per guidare la correzione.
- I tentativi ripetuti sono inefficaci quando l'informazione richiesta semplicemente non è presente nella fonte.
- Progettazione dei cicli di feedback: tracciare il pattern che ha generato una segnalazione (`detected_pattern`).
- Errori semantici (totali non coerenti) rispetto a errori sintattici (già gestiti tramite `tool_use`).

Competenze chiave

- Creare prompt di follow-up che includano il documento originale, un'estrazione errata e specifici errori di validazione.
- Riconoscere quando un nuovo tentativo sarà inefficace (ad esempio quando le informazioni richieste sono disponibili solo in un documento esterno).
- Includere campi `detected_pattern` nelle segnalazioni per analizzare i falsi positivi.
- Progettare meccanismi di autocorrezione estraendo sia `calculated_total` sia `stated_total` per individuare eventuali discrepanze.

4.5 Progettare Strategie Efficienti di Elaborazione Batch

Conoscenze chiave

- API Message Batches: riduzione dei costi del 50%, finestra di elaborazione fino a 24 ore e assenza di garanzie SLA sulla latenza.
- L'elaborazione batch è adatta a attività non bloccanti (report notturni, audit) e non è adatta a attività bloccanti (controlli pre-merge).
- L'API Batch non supporta chiamate a strumenti multi-turn all'interno della stessa richiesta.
- I campi `custom_id` consentono di correlare richieste e risposte all'interno dei batch.

Competenze chiave

- Utilizzare l'API sincrona per controlli bloccanti e l'API Batch per carichi di lavoro notturni o settimanali.
- Pianificare la frequenza di invio dei batch in base ai requisiti SLA (ad esempio finestre di 4 ore per garantire una copertura operativa di 30 ore con un tempo di elaborazione massimo di 24 ore).

- Gestire i fallimenti reinviando soltanto i documenti che hanno generato errori (identificati tramite `custom_id`).
- Perfezionare i prompt utilizzando un campione rappresentativo prima di avviare elaborazioni su larga scala.

4.6 Progettare Architetture di Revisione Multi-Istanza e Multi-Passaggio

Conoscenze chiave

- Limiti dell'autorevisione (self-review): il modello mantiene il proprio contesto di ragionamento ed è meno incline a mettere in discussione le proprie decisioni.
- Le istanze di revisione indipendenti (prive del contesto di generazione) sono più efficaci nell'individuare problemi sottili.
- Revisione multi-passaggio (multi-pass review): analisi locale per singolo file seguita da una fase di integrazione tra file per evitare la dispersione dell'attenzione.

Competenze chiave

- Utilizzare una seconda istanza indipendente di Claude per revisionare le modifiche senza il contesto della generazione originale.
- Suddividere le revisioni che coinvolgono più file in passaggi per singolo file, seguiti da passaggi di integrazione per analizzare i flussi di dati tra file.
- Utilizzare passaggi di verifica con autovalutazione del livello di confidenza per instradare le revisioni in modo calibrato e coerente.

Dominio 5: Gestione del Contesto e Affidabilità (15%)

5.1 Gestire il Contesto della Conversazione per Preservare le Informazioni Critiche

Conoscenze chiave

- Rischi della sintesi progressiva (progressive summarization): valori numerici, percentuali e date tendono a essere condensati in riepiloghi vaghi.
- Effetto "lost-in-the-middle": i modelli elaborano in modo affidabile l'inizio e la fine di input molto lunghi, ma possono trascurare informazioni presenti nella parte centrale.
- Gli output degli strumenti possono accumularsi nel contesto in modo sproporzionato rispetto alla loro rilevanza (ad esempio oltre 40 campi quando ne servono solo 5).
- L'importanza di inviare l'intera cronologia della conversazione nelle richieste API successive.

Competenze chiave

- Estrarre i fatti transazionali in un blocco persistente di "case facts" separato dalla cronologia sintetizzata.
- Ridurre gli output verbosi degli strumenti mantenendo solo i campi rilevanti.
- Posizionare le informazioni chiave all'inizio dei dati aggregati utilizzando intestazioni di sezione esplicite.
- Richiedere ai sottoagenti di includere metadati (date, fonti) negli output strutturati.

5.2 Progettare Modelli di Escalation Efficaci e Risolvere le Ambiguità

Conoscenze chiave

- Trigger appropriati per l'escalation: richiesta esplicita di un operatore umano, lacune o eccezioni nelle policy, impossibilità di progredire nella risoluzione del problema.
- Escalation immediata (richiesta esplicita di un umano) rispetto a tentativi di risoluzione autonoma (quando rientrano nell'ambito dell'agente).
- L'analisi del sentiment e le autovalutazioni del livello di confidenza del modello sono indicatori poco affidabili della complessità di un caso.
- In presenza di più corrispondenze possibili per un cliente, è necessario richiedere identificatori aggiuntivi anziché affidarsi a supposizioni euristiche.

Competenze chiave

- Definire criteri di escalation espliciti nel prompt di sistema, corredati da esempi few-shot.
- Eseguire immediatamente le richieste esplicite di assistenza umana senza effettuare ulteriori indagini.
- Attivare l'escalation quando la policy è ambigua o non fornisce indicazioni per una richiesta specifica.
- Richiedere identificatori aggiuntivi quando i risultati degli strumenti contengono più corrispondenze.

5.3 Implementare Strategie di Propagazione degli Errori nei Sistemi Multiagente

Conoscenze chiave

- Un contesto di errore strutturato (tipo di errore, query eseguita, risultati parziali, alternative possibili) consente al coordinatore di adottare strategie di recupero più intelligenti.
- È importante distinguere i problemi di accesso (ad esempio timeout che richiedono una decisione sul retry) dai risultati validamente vuoti (nessuna corrispondenza trovata).
- Stati di errore generici ("ricerca non disponibile") nascondono informazioni preziose al coordinatore.
- Sopprimere silenziosamente gli errori o interrompere l'intero workflow a causa di un singolo fallimento sono entrambi anti-pattern.

Competenze chiave

- Restituire un contesto di errore strutturato che includa: tipo di errore, operazione tentata, risultati parziali e possibili alternative.
- Distinguere chiaramente i problemi di accesso dai risultati validamente vuoti.
- Effettuare il recupero locale nei sottoagenti per errori transitori; propagare soltanto gli errori non recuperabili insieme agli eventuali risultati parziali.
- Annotare il livello di copertura nella sintesi finale, indicando cosa è supportato da evidenze solide e dove permangono lacune informative.

5.4 Gestire il Contesto in Modo Efficiente Durante l'Analisi di Codebase di Grandi Dimensioni

Conoscenze chiave

- Degrado del contesto nelle sessioni lunghe: il modello inizia a produrre risposte meno stabili e a fare riferimento a "pattern tipici" invece che a classi o componenti specifici.
- I file scratchpad consentono di preservare le scoperte e le informazioni chiave oltre i limiti del contesto disponibile.

- La delega ai sottoagenti permette di isolare l'output dettagliato delle attività di esplorazione.
- La persistenza strutturata dello stato consente il recupero del lavoro in caso di interruzioni o crash.

Competenze chiave

- Creare sottoagenti dedicati a domande specifiche mantenendo il coordinamento generale nell'agente principale.
- Utilizzare file scratchpad per memorizzare le scoperte principali e farvi riferimento successivamente.
- Riassumere le informazioni chiave prima di avviare sottoagenti per le fasi successive dell'analisi.
- Utilizzare `/compact` per ridurre il consumo di contesto durante indagini prolungate.

5.5 Progettare Workflow con Supervisione Umana e Calibrazione della Confidenza

Conoscenze chiave

- Le metriche aggregate (ad esempio il 97% di accuratezza complessiva) possono nascondere prestazioni scarse su specifici tipi di documenti o campi.
- Il campionamento casuale stratificato (stratified random sampling) consente di misurare i tassi di errore anche nelle estrazioni ad alta confidenza.
- Calibrazione della confidenza a livello di campo tramite set di validazione etichettati.
- Prima di automatizzare un processo, è necessario validarne l'accuratezza per tipologia di documento e segmento di dati.

Competenze chiave

- Implementare il campionamento casuale stratificato per individuare nuovi pattern di errore.
- Analizzare l'accuratezza per tipologia di documento e per campo, verificando che le prestazioni rimangano stabili.
- Generare punteggi di confidenza a livello di singolo campo e calibrare le soglie di revisione utilizzando dati etichettati.
- Instradare verso revisione umana le estrazioni a bassa confidenza o provenienti da fonti ambigue.

5.6 Preservare la Provenienza delle Informazioni e Gestire l'Incertezza nella Sintesi da Più Fonti

Conoscenze chiave

- L'attribuzione delle informazioni viene persa durante la sintesi se non vengono mantenute le associazioni "affermazione → fonte".
- Le associazioni strutturate tra affermazioni e fonti devono essere preservate durante l'aggregazione.
- In presenza di statistiche contrastanti, è necessario annotare il conflitto con le relative attribuzioni invece di scegliere arbitrariamente un valore.
- Includere le date di pubblicazione o raccolta dei dati evita di interpretare erroneamente differenze temporali come contraddizioni.

Competenze chiave

- Richiedere ai sottoagenti di produrre associazioni "affermazione → fonte" (URL, nome del documento, citazioni).

- Strutturare i report separando chiaramente i risultati consolidati da quelli controversi o contestati.
- Conservare valori contrastanti con annotazioni esplicative e inoltrarli al coordinatore per la riconciliazione.
- Includere le date di pubblicazione per consentire una corretta interpretazione temporale delle informazioni.
- Presentare i contenuti in base alla loro tipologia: dati finanziari in tabelle, notizie in forma discorsiva, risultati tecnici come elenchi strutturati.

Esercizi Pratici

Esercizio 1: Agente Multi-strumento con Logica di Escalation

Obiettivo: Progettare un ciclo agente con integrazione degli strumenti, gestione strutturata degli errori ed escalation.

Passaggi:

1. Definire 3–4 strumenti MCP con descrizioni dettagliate (includere due strumenti simili per testare la selezione degli strumenti).
2. Implementare un ciclo agente che controlli `stop_reason` (`"tool_use"` / `"end_turn"`).
3. Aggiungere risposte di errore strutturate: `errorCategory`, `isRetryable`, descrizione.
4. Implementare un hook intercettore che blocchi le operazioni sopra una certa soglia e le instradi verso l'escalation.
5. Testare con richieste multi-aspetto.

Domini: 1 (Architettura degli agenti), 2 (Strumenti e MCP), 5 (Contesto e affidabilità)

Esercizio 2: Configurare Claude Code per lo Sviluppo in Team

Obiettivo: Configurare CLAUDE.md, comandi personalizzati, regole specifiche per percorso e server MCP.

Passaggi:

1. Creare un CLAUDE.md a livello progetto con standard universali.
2. Creare file in `.claude/rules/` con frontmatter YAML per diverse aree del codice (`paths: ["src/api/**/*"]`, `paths: ["**/*.test.*"]`).
3. Creare una skill di progetto in `.claude/skills/` con `context: fork` e `allowed-tools`.
4. Configurare un server MCP in `.mcp.json` usando variabili d'ambiente e un override personale in `~/.claude.json`.
5. Testare la modalità di pianificazione rispetto all'esecuzione diretta su attività di diversa complessità.

Domini: 3 (Configurazione di Claude Code), 2 (Strumenti e MCP)

Esercizio 3: Pipeline di Estrazione di Dati Strutturati

Obiettivo: Schemi JSON, `tool_use` per output strutturato, cicli di validazione/retry, elaborazione batch.

Passaggi:

1. Definire uno strumento di estrazione con schema JSON (campi obbligatori/facoltativi, enum con `"other"`, campi nullable).
2. Costruire un ciclo di validazione: in caso di errore, riprovare includendo il documento, l'estrazione errata e lo specifico errore di validazione.
3. Aggiungere esempi few-shot per documenti con strutture diverse.
4. Utilizzare l'elaborazione batch tramite API Message Batches: 100 documenti, gestendo i fallimenti tramite `custom_id`.
5. Instradare verso revisione umana: punteggi di confidenza a livello di campo, analisi per tipo di documento.

Domini: 4 (Prompt engineering), 5 (Contesto e affidabilità)

Esercizio 4: Progettare e Debuggare una Pipeline di Ricerca Multiagente

Obiettivo: Orchestrazione dei sottoagenti, passaggio del contesto, propagazione degli errori, sintesi con tracciamento delle fonti.

Passaggi:

1. Un coordinatore con 2+ sottoagenti (`allowedTools` include `"Task"`, il contesto viene passato esplicitamente nei prompt).
2. Eseguire i sottoagenti in parallelo tramite più chiamate `Task` in una singola risposta.
3. Richiedere output strutturato dai sottoagenti: affermazione, citazione, URL della fonte, data di pubblicazione.
4. Simulare un timeout di un sottoagente: restituire al coordinatore un contesto di errore strutturato e continuare con risultati parziali.
5. Testare con dati conflittuali: preservare entrambi i valori con attribuzione; separare risultati confermati e risultati contestati.

Domini: 1 (Architettura degli agenti), 2 (Strumenti e MCP), 5 (Contesto e affidabilità)

Appendice: Tecnologie e Concetti

Tecnologia	Aspetti principali
Claude Agent SDK	AgentDefinition, cicli degli agenti, `stop_reason`, hook (PostToolUse), creazione di sottoagenti tramite Task, `allowedTools`
Model Context Protocol (MCP)	Server MCP, strumenti, risorse, `isError`, descrizioni degli strumenti, `.mcp.json`, variabili d'ambiente
Claude Code	Gerarchia di CLAUDE.md, `.claude/rules/` con pattern glob, `.claude/commands/`, `.claude/skills/` con SKILL.md, modalità di pianificazione (*planning mode*), `/compact`, `--resume`, `fork_session`
Claude Code CLI	`-p` / `--print` per la modalità non interattiva, `--output-format json`, `--json-schema`

Tecnologia	Aspetti principali
Claude API	`tool_use` con schemi JSON, `tool_choice` ("auto"/"any"/forzato), `stop_reason`, `max_tokens`, prompt di sistema
Message Batches API	Risparmio del 50%, finestra di elaborazione fino a 24 ore, `custom_id`, nessun supporto per chiamate multi-turno agli strumenti
JSON Schema	Campi obbligatori vs facoltativi, campi nullable, tipi enum, `"other"` + campo dettaglio, modalità strict
Zod	Validazione degli schemi, errori semantici, cicli di validazione/retry
Strumenti integrati	Read, Write, Edit, Bash, Grep, Glob — scopo e criteri di selezione
Few-shot prompting	Esempi mirati per situazioni ambigue, generalizzazione a nuovi pattern
Prompt chaining	Scomposizione sequenziale in passaggi focalizzati
Finestra di contesto	Budget di token, sintesi progressiva, fenomeno del "lost in the middle", file scratchpad
Gestione delle sessioni	Resume, `fork_session`, sessioni nominate, isolamento del contesto
Calibrazione della confidenza	Punteggi a livello di campo, calibrazione su dataset etichettati, campionamento stratificato

Argomenti Fuori Programma

I seguenti argomenti correlati **NON** saranno presenti all'esame:

- Fine-tuning dei modelli Claude o addestramento di modelli personalizzati
- Autenticazione delle API Claude, fatturazione o gestione degli account
- Implementazioni dettagliate in linguaggi di programmazione o framework specifici (oltre quanto necessario per la configurazione di strumenti e schemi)
- Distribuzione o hosting di server MCP (infrastruttura, networking, orchestrazione di container)
- Architettura interna di Claude, processo di addestramento o pesi del modello
- Constitutional AI, RLHF o metodologie di addestramento per la sicurezza
- Modelli di embedding o dettagli implementativi di database vettoriali
- Utilizzo del computer (*computer use*: automazione del browser, interazione desktop)
- Capacità di analisi delle immagini (Vision)
- Streaming API o Server-Sent Events (SSE)
- Rate limiting, quote o calcoli dettagliati dei costi API
- OAuth, rotazione delle API key o dettagli sui protocolli di autenticazione
- Configurazioni specifiche per provider cloud (AWS, GCP, Azure)
- Benchmark prestazionali o metriche di confronto tra modelli
- Dettagli implementativi della prompt caching (oltre al sapere che esiste)
- Algoritmi di conteggio dei token o dettagli della tokenizzazione

Raccomandazioni per la Preparazione

1. **Costruisci un agente con il Claude Agent SDK** — implementa un ciclo agente completo con chiamate agli strumenti, gestione degli errori e gestione delle sessioni. Esercitati con i sottoagenti e con il passaggio esplicito del contesto.
2. **Configura Claude Code per un progetto reale** — utilizza la gerarchia di CLAUDE.md, regole specifiche per percorso in `.claude/rules/`, skill con `context: fork` e `allowed-tools`, e integrazione di server MCP.
3. **Progetta e testa strumenti MCP** — scrivi descrizioni che distinguano chiaramente strumenti simili, restituiscano errori strutturati con categorie e flag di retry e testali con richieste utente ambigue.
4. **Costruisci una pipeline di estrazione dati** — utilizza `tool_use` con schemi JSON, cicli di validazione/retry, campi opzionali/nullable e l'elaborazione batch tramite la Message Batches API.
5. **Esercitati con il prompt engineering** — aggiungi esempi few-shot per scenari ambigui, criteri di revisione espliciti e architetture multi-pass per revisioni di codice su larga scala.
6. **Studia i pattern di gestione del contesto** — estrai i fatti importanti da output verbosi, utilizza file scratchpad e delega l'attività esplorativa ai sottoagenti per gestire i limiti di contesto.
7. **Comprendi escalation e supervisione umana (*human-in-the-loop*)** — quando effettuare un'escalation (lacune nelle policy, richiesta esplicita di un operatore umano, impossibilità di progredire) e come progettare flussi di instradamento basati sulla confidenza.
8. **Sostieni un esame di prova** prima di affrontare quello reale. Utilizza gli stessi scenari e lo stesso formato dell'esame ufficiale.